Objektorientierte Programmierung

WOCHE 2:
DESIGN PATTERNS I – STRUKTUR UND ERZEUGUNG

PETER BERNHARDT

März 2026

1

Teil 1: Einführung in Design Patterns

2

Willkommen zu Woche 2

Nachdem wir uns in der ersten Woche mit Clean Code beschäftigt haben, gehen wir nun einen Schritt weiter: zum Software-Design. Wir schauen uns an, wie man architektonische Probleme löst, die immer wieder auftreten.

Lernziele:

Das Konzept der Design Patterns verstehen.

Erzeugungsmuster (Singleton, Factory Method) anwenden.

Strukturmuster (Adapter, Decorator) in der Praxis nutzen.

3

Was sind Design Patterns?

Design Patterns (Entwurfsmuster) sind bewährte Lösungsschablonen für wiederkehrende Probleme im Software-Design.

Sie wurden maßgeblich durch die “Gang of Four” (GoF) im Jahr 1994 formalisiert.

Es sind keine fertigen Code-Schnipsel, sondern abstrakte Konzepte.

Sie bilden ein gemeinsames Vokabular für Entwickler.

4

Warum Design Patterns in der Verwaltung?

Gerade in der Verwaltungsinformatik baut man Systeme für die “Ewigkeit”.

Lange “Lebensdauer: Begrenzte Mittel bedingen lange Lebenszyklen.

Gesetzesänderungen: Algorithmen ändern sich, das System muss flexibel bleiben.

Legacy-Systeme: Wir müssen oft alte mit neuen Systemen verbinden.

Entwurfsmuster helfen uns, robuste, erweiterbare und wartbare Architekturen zu schaffen.

5

Drei Kategorien von Mustern

Die GoF unterteilt Muster in drei Hauptkategorien:

1. **Erzeugungsmuster (Creational):** Wie werden Objekte flexibel erzeugt?
2. **Strukturmuster (Structural):** Wie werden Klassen und Objekte zu größeren Strukturen zusammengefügt?
3. **Verhaltensmuster (Behavioral):** Wie kommunizieren Objekte effizient miteinander?

6

Keine “Copy-Paste” Lösungen

Wichtig: Ein Design Pattern ist keine Bibliothek, die man einbindet.

Sie müssen das Muster an Ihre spezifische Domäne (z.B. das Meldewesen) anpassen.

Setzen Sie Muster nur ein, wenn Sie das entsprechende Problem haben (Vermeidung von Over-Engineering!).

Die Abgenzung der Muster ist nicht immer einfach. Es sind keine “Schablonen”…

7

Teil 2: Das Singleton Pattern

8

Einführung: Singleton

Das Singleton-Muster gehört zu den *Erzeugungsmustern*.

Es stellt sicher, dass von einer Klasse **genau eine einzige Instanz** existiert und bietet einen globalen Zugriffspunkt darauf.

Problemstellung

Oft gibt es Ressourcen in einem System, die nur einmal vorhanden sein dürfen. Wenn Sie beispielsweise eine Konfigurationsdatei (Datenbankanbindung, etc.) nutzen, wollen Sie nicht, dass 10 verschiedene Module 10 verschiedene Kopien dieser Konfiguration im Speicher halt

9

Anwendung in der Verwaltung (Konfigurationsmanager)

Stellen Sie sich ein Fachverfahren vor. Es gibt zentrale Einstellungen:

- URL des zentralen Melderegisters
- Aktuelle Mehrwertsteuersätze
- Name der ausführenden Behörde

Ein `KonfigurationsManager` als Singleton stellt sicher, dass alle Module exakt denselben Stand dieser Daten nutzen.

10

Singleton: Die Eigenschaften

Wie erreicht man, dass eine Klasse nur einmal instanziiert werden kann?

1. Ein **privater Konstruktor** verhindert, dass jemand von außen `new` aufruft.
2. Eine **statische Variable** hält die einzige Instanz.
3. Eine **öffentliche statische Eigenschaft** liefert diese Instanz zurück.

11

Singleton: Code (Felder & Konstruktor)

```
public sealed class KonfigurationsManager
{
    private static KonfigurationsManager _instance;
    private static readonly object _lock = new object();

    // Privater Konstruktor verhindert Instanziierung von außen
    private KonfigurationsManager()
    {
        RegisterUrl = "https://zentralregister.behoerde.de/api";
        BehoerdenName = "Landesamt für Digitalisierung";
    }

    public string RegisterUrl { get; set; }
    public string BehoerdenName { get; set; }

    // ...

}
```

12

Singleton: Code (Thread-Safe Instance)

```
public sealed class KonfigurationsManager
{
    // ...

    public static KonfigurationsManager Instance
    {
        get
        {
            // Thread-safe Implementierung (Double-Check Locking)
            if (_instance == null)
            {
                lock (_lock)
                {
                    if (_instance == null)
                    {
                        _instance = new KonfigurationsManager();
                    }
                }
            }
            return _instance;
        }
    }
}
```

13

Kritik am Singleton

Das Singleton wird oft als “Anti-Pattern” bezeichnet.

Warum?

Es führt einen **globalen Zustand** (Global State) ein, der von überall geändert werden kann.

Es **verdeckt** Abhängigkeiten: Eine Klasse, die das Singleton nutzt, zeigt dies nicht in ihrem Konstruktor.

Es erschwert das **Unit Testing** (Mocking), da man die feste Instanz nur schwer durch einen Fake ersetzen kann.

14

Wann man es (wirklich) nutzen sollte

Nutzen Sie Singleton nur, wenn die Instanzierung wirklich systemweit streng limitiert sein **muss** (z.B. Hardware-Treiber, Logging-Dienste in kleinen Systemen).

In modernen Web-Anwendungen überlässt man diese Aufgabe oft dem “Dependency Injection Container” (dazu mehr in Woche 4).

15

Teil 3: Factory Patterns

16

Einführung: Factory Method

Auch dieses Muster gehört zu den *Erzeugungsmustern*.

Das Factory Method Pattern definiert ein Interface zur Erstellung eines Objekts.

Die Entscheidung, **welche** konkrete Klasse instanziiert wird, wird jedoch an die Unterklassen delegiert.

17

Problemstellung: Harte Verdrahtung

Wenn Sie überall im Code `new BewilligungsBescheid()` schreiben, koppeln Sie Ihre Logik eng an diese konkrete Klasse. Was passiert, wenn sich der Prozess zur Erstellung ändert oder neue Bescheid-Arten oder -Varianten hinzukommen? Der Code wird unflexibel. Eine **lose Kopplung* ist die Lösung.

18

Anwendung: Erstellung von Bescheid-Typen

Ein Fachverfahren muss verschiedene Arten von Bescheiden erzeugen:

- Bewilligungsbescheid
- Ablehnungsbescheid
- Gebührenbescheid

Der Versandprozess (Drucken, Kuvertieren, Porto) ist bei allen gleich, nur der Inhalt variiert.

Es können jedoch in Zukunft weitere Bescheide hinzukommen.

19

Factory Method: Produkt-Klassen (Code)

Wir definieren ein abstraktes Produkt und konkrete Implementierungen:

```
// Das Produkt-Interface / abstrakte Basisklasse
public abstract class Bescheid
{
    public abstract void GeneriereInhalt();
}

// Konkrete Produkte
public class BewilligungsBescheid : Bescheid
{
    public override void GeneriereInhalt()
        => Console.WriteLine("Ihr Antrag wurde bewilligt.");
}

public class AblehnungsBescheid : Bescheid
{
    public override void GeneriereInhalt()
        => Console.WriteLine("Ihr Antrag wurde leider abgelehnt.");
}
```

20

Factory Method: Die abstrakte Factory (Code)

Die Kernlogik arbeitet nur mit der Abstraktion, nicht mit den konkreten Typen.

```
// Die Factory (Erzeuger)
public abstract class BescheidErzeuger
{
    // Die eigentliche Factory Method
    public abstract Bescheid ErstelleBescheid();

    // Die Kernlogik (für alle gleich!)
    public void SendeBescheid()
    {
        var bescheid = ErstelleBescheid();
        bescheid.GeneriereInhalt();
        Console.WriteLine("Bescheid wird per Post/E-Mail versendet.");
    }
}
```

21

Factory Method: Konkrete Erzeuger (Code)

Die Entscheidung der Instanziierung liegt in diesen kleinen Unterklassen.

```
// Konkrete Erzeuger
public class BewilligungsErzeuger : BescheidErzeuger
{
    public override Bescheid ErstelleBescheid()
        => new BewilligungsBescheid();
}

public class AblehnungsErzeuger : BescheidErzeuger
{
    public override Bescheid ErstelleBescheid()
        => new AblehnungsBescheid();
}
```

22

Factory Method: Hauptprogramm (Code)

Das Hauptprogramm “kennt” nun nur noch die Erzeuger nicht mehr die Bescheide.

```
class Program
{
    static void Main()
    {
        BescheidErzeuger erzeuger;

        erzeuger = new BewilligungsErzeuger();
        erzeuger.SendeBescheid();

        Console.WriteLine();

        erzeuger = new AblehnungsErzeuger();
        erzeuger.SendeBescheid();
    }
}
```

23

Vorteile und Nachteile der Factory Method

Vorteile

- **Single Responsibility:** Die Erzeugung der Objekte ist vom Rest des Codes isoliert.
- **Open-Closed Principle:** Sie können neue Bescheid-Arten (z.B. `AnhoerungsBescheid`) hinzufügen, ohne den bestehenden Versand-Code zu verändern.

Nachteile

- **Schlechte Erweiterbarkeit:** Das Hauptprogramm muss immer noch alle konkreten `BescheidErzeuger` kennen.

Auftritt **Simple Factory**!

24

Simple Factory: Die Fabrik (Code)

Die Fabrik wählt den richtigen Erzeuger anhand eines Parameters (hier ein einfacher String) aus.

In der Praxis ist die häufig ein Enum.

```
public static class ErzeugerFactory
{
    public static BescheidErzeuger Erstelle(string typ)
    {
        return typ.ToLower() switch
        {
            "bewilligung" => new BewilligungsErzeuger(),
            "ablehnung"   => new AblehnungsErzeuger(),
            _ => throw new ArgumentException("Unbekannter Typ")
        };
    }
}
```

25

Simple Factory: Altes vs. Neues C#

```
public static class ErzeugerFactory
{
    public static BescheidErzeuger Erstelle(string typ)
    {
        return typ.ToLower() switch
        {
            "bewilligung" => new BewilligungsErzeuger(),
            "ablehnung"   => new AblehnungsErzeuger(),
            _ => throw new ArgumentException("Unbekannter Typ")
        };

        // synonym zu:
        switch(typ.ToLower())
        {
            case "bewilligung":
                return new BewilligungsErzeuger();
            case "ablehnung":
                return new AblehnungsErzeuger();
            default:
                throw new ArgumentException("Unbekannter Typ");
        }
    }
}
```

26

Simple Factory: Das Hauptprogramm (Code)

Das Hauptprogramm kennt nur noch den abstrakten **`Bescheiderzeuger`** und wählt den konkreten mit Hilfe eines Parameters (hier Namen) aus.

```
class Program
{
    static void Main()
    {
        BescheidErzeuger erzeuger;

        erzeuger = ErzeugerFactory.Erstelle("bewilligung");
        erzeuger.SendeBescheid();

        erzeuger = ErzeugerFactory.Erstelle("ablehnung");
        erzeuger.SendeBescheid();
    }
}
```

27

Vorteile der Simple Factory

Weniger Wissen im Hauptprogramm: Das Hauptprogramm hat eine losere Kopplung an konkrete Klassen.

Zentrale Auswahl-Logik: Die Entscheidung „welcher Erzeuger für welchen Fall?“ liegt an einer Stelle.

Einfache Mapping-Änderungen: Wenn sich die Zuordnung ändert, ändert man nur die Fabrik.

28

Unterscheidung: Factory Method + Simple Factory

Factory Method: Wie entsteht ein Produkt?

Simple Factory: Welcher Erzeuger wird gewählt?

Wichtige Einschränkung:

Die `Simple Factory` selbst verletzt oft das Open-Closed-Prinzip, weil sie beim Hinzufügen neuer Typen angepasst werden muss (`switch` wächst).

29

Teil 4: Adapter Pattern

30

Einführung: Adapter Pattern

Der Adapter ist ein Strukturmuster.

Er erlaubt es Objekten mit inkompatiblen Schnittstellen (Interfaces), zusammenzuarbeiten.

Er verhält sich wie ein Reiseadapter für Steckdosen: Er macht den fremden Stecker passend für die lokale Infrastruktur.

31

Das Problem: Legacy-Systeme

In der Verwaltung können Sie Altsysteme nicht einfach abschalten.

Sie müssen moderne Portale mit 20 Jahre alten Mainframes verbinden.

Beispiel: Anbindung eines Zentralregisters

Wir haben ein Web-Portal, das eine `Person` benötigt.

Das Legacy-Zentralregister hat keine moderne REST-API, sondern nur eine Funktion, die einen String wie `"Mustermann;Max;1980-01-01"` zurückgibt.

32

Adapter: Das Ziel-Interface (Code)

So möchte unser modernes System arbeiten:

```
public class Person {
    public string Vorname { get; set; }
    public string Nachname { get; set; }
    public DateTime Geburtsdatum { get; set; }
}

// Bestehendes, modernes System erwartet dieses Interface
public interface IPersonenQuelle
{
    Person GetPerson(string id);
}
```

33

Adapter: Das alte System (Code)

Die inkompatible Klasse, die wir nicht ändern können oder dürfen:

```
// Das alte Legacy-System (inkompatibel)
public class AltesRegisterSystem
{
    public string SuchePersonDatensatz(string personalId)
    {
        // Simulation eines alten Datenbank-Aufrufs
        // Liefert z.B. "Mustermann;Max;1980-01-01"
        return "Mustermann;Max;1980-01-01";
    }
}
```

34

Adapter: Die Implementierung (Code)

Der Adapter übersetzt zwischen den Welten:

```
// Der Adapter schließt die Lücke
public class RegisterAdapter : IPersonenQuelle
{
    private readonly AltesRegisterSystem _legacySystem = new
                                                AltesRegisterSystem();

    public Person GetPerson(string id)
    {
        string rohDaten = _legacySystem.SuchePersonDatensatz(id);
        string[] teile = rohDaten.Split(';');

        return new Person {
            Nachname = teile[0],
            Vorname = teile[1],
            Geburtsdatum = DateTime.Parse(teile[2])
        };
    }
}
```

35

Adapter: Vorteile

Integration bestehender Systeme: Alte und neue Komponenten können zusammenarbeiten, obwohl ihre Schnittstellen nicht zusammenpassen.

Lose Kopplung: Das moderne System bleibt völlig unabhängig von den Eigenheiten des Legacy-Systems.

Austauschbarkeit: Wenn das alte Register irgendwann durch ein neues ersetzt wird, müssen wir nur einen neuen Adapter schreiben – die Fachlogik bleibt unberührt.

Kapselung technischer Details: Formatumwandlungen und Übersetzungslogik bleiben an einer zentralen Stelle.

36

Adapter: Nachteile

Zusätzliche Komplexität: Es kommt eine weitere Klasse zwischen Fachlogik und bestehendem System hinzu.

Leistungs-Overhead: Die zusätzliche Übersetzungsschicht kostet etwas Laufzeit und Speicher.

Verdeckt Schwächen des Altsystems nur: Der Adapter löst nicht die eigentlichen Probleme des Legacy-Systems, sondern kapselt sie nur.

37

Teil 5: Decorator Pattern

38

Einführung: Decorator Pattern

Der Decorator ist ein Strukturmuster, das es erlaubt, einem Objekt dynamisch zusätzliches Verhalten hinzuzufügen.

Es ist eine flexible Alternative zur klassischen Vererbung.

39

Das Problem der Klassenexplosion

Angenommen, Sie haben Basis-Anträge und möchten zwei Zusatzoptionen anbieten. Wenn Sie Vererbung nutzen, brauchen Sie für jede Kombination eine Klasse:

```
Reisepass
ReisepassMitExpress
ReisepassMit48Seiten
ReisepassMitExpressUnd48Seiten
```

Das führt schnell zu vielen Klassen! (2^n)

Wenn man also irgendwann auf die brillante Idee kommt, 10 Optionen anzubieten, hat man 1024 Kombinationen.

Viel Spaß beim Testen.

40

Anwendung: Antrags-Zusatzleistungen

In der Verwaltung können Bürger oft Optionen (Express-Zuschlag, Zustellung per Post) dazubuchen.

Statt die Vererbungshierarchie aufzublähen, “dekorieren” (umhüllen) wir das Basis-Objekt wie eine Zwiebel.

41

Decorator: Basis und Komponente (Code)

```
// Das Basis-Interface
public interface IAntrag
{
    double BerechneKosten();
    string GetBeschreibung();
}

// Die konkrete Basis-Komponente
public class ReisepassAntrag : IAntrag
{
    public double BerechneKosten() => 60.00;
    public string GetBeschreibung() => "Reisepass (Standard)";
}
```

42

Decorator: Die Basis-Decorator-Klasse (Code)

Diese Klasse leitet die Aufrufe standardmäßig an das umhüllte Objekt (`_antrag`) weiter.

```
// Basis-Decorator
public abstract class AntragDecorator : IAntrag
{
    protected IAntrag _antrag;

    public AntragDecorator(IAntrag antrag) => _antrag = antrag;

    public virtual double BerechneKosten()
        => _antrag.BerechneKosten();

    public virtual string GetBeschreibung()
        => _antrag.GetBeschreibung();
}
```

43

Decorator: Der konkrete Express-Decorator (Code)

Hier fügen wir das neue Verhalten hinzu:

```
// Konkreter Decorator: Express Option
public class ExpressOption : AntragDecorator
{
    public ExpressOption(IAntrag antrag) : base(antrag) { }

    public override double BerechneKosten()
        => base.BerechneKosten() + 32.00; // Zuschlag

    public override string GetBeschreibung()
        => base.GetBeschreibung() + " + Express-Zuschlag";
}
```

44

Decorator: Nutzung des Decorators

So nutzen wir den Decorator:

```
class Program
{
    static void Main()
    {
        // 1. Basis erstellen (60 EUR)
        IAntrag meinAntrag = new ReisepassAntrag();

        // 2. Mit Express dekorieren (+32 EUR)
        meinAntrag = new ExpressOption(meinAntrag);

        Console.WriteLine(meinAntrag.GetBeschreibung());
        // Ausgabe: Reisepass (Standard) + Express-Zuschlag


        Console.WriteLine(meinAntrag.BerechneKosten());
        // Ausgabe: 92.00

    }
}
```

45

Decorator: Der konkrete 48 Seiten-Decorator (Code)

Hier fügen wir eine neue Option hinzu:

```
// Konkreter Decorator: Express Option
public class MehrSeitenOption : AntragDecorator
{
    public MehrSeitenOption(IAntrag antrag) : base(antrag) { }

    public override double BerechneKosten()
        => base.BerechneKosten() + 22.00; // Zuschlag

    public override string GetBeschreibung()
        => base.GetBeschreibung() + " + 48Seiten-Zuschlag";
}
```

46

Decorator: Verschachtelung von Decoratoren

So setzen wir eine Zwiebel zusammen:

```
class Program
{
    static void Main()
    {
        // 1. Basis erstellen (60 EUR)
        IAntrag meinAntrag = new ReisepassAntrag();

        // 2. Mit Express dekorieren (+32 EUR)
        meinAntrag = new ExpressOption(meinAntrag);

        // 3. Mit 48Seiten dekorieren (+22 EUR)
        meinAntrag = new MehrSeitenOption(meinAntrag);

        Console.WriteLine(meinAntrag.GetBeschreibung());
        // Ausgabe: Reisepass (Standard) + Express-Zuschlag +
                    48Seiten-Zuschlag

        Console.WriteLine(meinAntrag.BerechneKosten());
        // Ausgabe: 114.00

    }
}
```

Sie können Optionen beliebig ineinander verschachteln!

47

Decorator: Vorteile

Flexible Erweiterung zur Laufzeit: Verhalten kann dynamisch hinzugefügt oder kombiniert werden.

Vermeidet Klassenexplosion: Statt vieler Unterklassen werden Optionen durch Kombination von Decorators aufgebaut.

Open-Closed-Prinzip: Neue Optionen können ergänzt werden, ohne bestehende Klassen zu ändern.

Single Responsibility: Jede Zusatzfunktion (z.B. Express, 48 Seiten) ist in einer eigenen Klasse gekapselt.

Kombinierbarkeit: Mehrere Decorators lassen sich in beliebiger Reihenfolge verschachteln.

48

Decorator: Nachteile

Mehr Komplexität im Aufbau: Viele kleine Klassen und verschachtelte Objekte erschweren den Überblick.

Reihenfolgeabhängigkeit möglich: Unterschiedliche Decorator-Reihenfolgen können zu anderem Ergebnis führen.

Debugging aufwendiger: Verhalten verteilt sich über mehrere Ebenen der Dekoration.

Objektidentität/Transparenz: Gerade bei unbekanntem Code ist nicht immer klar, welches „echte“ Objekt gerade verwendet wird.

`Decorator` ist ideal für optionale, kombinierbare Zusatzfunktionen; bei hoher Verschachtelung steigt jedoch die kognitive Komplexität.

49

Teil 6: Zusammenfassung

50

Wrap-up Woche 2

Singleton: Wenn es garantiert nur ein Objekt geben darf (Vorsicht: Anti-Pattern Gefahr!).

Factory's: Delegation der Objekterzeugung an Unterklassen (für Flexibilität).

Adapter: Der Übersetzer zwischen inkompatiblen Welten (Legacy-Integration).

Decorator: Dynamisches Hinzufügen von Funktionen ohne Klassenexplosion.

51

Ausblick auf nächste Woche

Diese Woche haben wir Objekte erzeugt (Creational) und strukturiert (Structural).

Nächste Woche betrachten wir die dritte Kategorie: **Verhaltensmuster (Behavioral Patterns)**, wie das Strategy- und Observer-Pattern.

52